Simulação e Teste de Software

Laboratório 8: Critérios de Cobertura Estrutural

Prof. Luciano Rossi

Departamento de Ciência da Computação
Centro Universitário FEI

2º Semestre de 2026

Laboratório 8

Contexto

Objetivo desta aula

Aplicar os critérios da aula 8 a código com defeitos: montar a base de caminhos de uma função com laço, comparar conjuntos MC/DC e encontrar pares definição-uso que uma suíte com 100% de ramos não percorre.

Em todos os exercícios, os casos de teste são implementados com pytest. A entrega segue como nos laboratórios anteriores.

Preparação do Ambiente: Linux

centro universitário FEI

</> Código

Crie uma pasta `lab08` e, dentro dela, o ambiente virtual e as ferramentas do laboratório:

Terminal

```
python3 -m venv .venv
source .venv/bin/activate
pip install pytest==9.1.1 coverage==7.16.2 pytest-cov==7.1.0
```

Cada exercício tem um arquivo de código e um de testes (`test_*.py`); com o ambiente ativo, rode tudo com `pytest -q`.

Preparação do Ambiente: Windows

centro universitário FEI

</> Código

Os mesmos passos, no PowerShell:

PowerShell

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install pytest==9.1.1 coverage==7.16.2 pytest-cov==7.1.0
```

- No Prompt de Comando (cmd), a ativação é `.venv\Scripts\activate.bat`
- Se o PowerShell bloquear a ativação, rode antes `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
- Com o ambiente ativo, rode os testes com `pytest -q`, como no Linux

Dicas: Comandos de Cobertura

centro universitário FEI

Código

Terminal

```
# so comandos (C0), relatorio do modulo senha
coverage run --source=senha -m pytest -q
coverage report -m
# comandos e ramos (C1)
coverage run --branch --source=senha -m pytest -q
coverage report -m
# o mesmo pelo pytest-cov, em um comando
pytest -q --cov=senha --cov-branch --cov-report=term-missing
```

- Na coluna `Missing`, `5->7` indica um ramo não percorrido: a passagem da linha 5 para a 7
- A contagem de ramos varia entre versões; por isso a preparação instala as mesmas do gabarito

Caminhos e Laços



Força da Senha

Código do Exercício 1

senha.py

```python
def forca(senha):
    if len(senha) < 8:
        return "fraca"
    pontos = 0
    for c in senha:
        if c.isdigit():
            pontos += 1
        elif not c.isalnum():
            pontos += 2
    if pontos >= 4:
        return "forte"
    return "media"
```

Dígito vale 1 ponto e símbolo vale 2; letras não pontuam.

Força da Senha

Exercício 1

a) Desenhe o GFC de `forca` e calcule $V(G)$ pelas duas fórmulas

b) Monte uma base de $V(G)$ caminhos linearmente independentes, com entrada e saída esperada, e justifique a independência de cada um. Algum caminho é inviável? Por quê?

c) Implemente os caminhos viáveis em um teste com `parametrize` e meça com pytest-cov e `--cov-branch`. Quantos testes deram 100% de ramos, comparados com $V(G)$?

d) Aplique o teste de laços ao `for`: quantas voltas são possíveis? Acrescente os casos de laço que fazem sentido

Condições e MC/DC



Controle de Embarque

Código do Exercício 2

Especificação: embarca quem tem cartão de embarque e documento válido, e cujo grupo já foi chamado ou que tem prioridade.

embarque.py

```python
def pode_embarcar(cartao, documento, grupo_chamado, prioridade):
    if cartao and documento and grupo_chamado or prioridade:
        return True
    return False
```

Na aula 8, o MC/DC nasceu na aviação; aqui ele confere a regra do portão de embarque.

Controle de Embarque

Exercício 2

a) Em `mcdc.py`, escreva `regra(...)`, direto da especificação, e `pares_independentes(decisao, n)`, que devolve, para cada condição, os pares de linhas da tabela-verdade que diferem só nela e mudam a decisão

b) Quantos pares tem cada condição? Quantos conjuntos MC/DC mínimos existem, e com quantos casos?

c) Compare `pode_embarcar` com `regra` em dois testes: todos V e todos F. Meça os ramos

d) Compare de novo em cada conjunto MC/DC mínimo. Quais acham o defeito? Explique, e explique por que o MCC sempre acharia. Corrija a função

Fluxo de Dados



Preço com Desconto VIP

</> Código do Exercício 3

Especificação: cliente VIP tem 20% de desconto; todo pedido custa no mínimo R$50.

preco.py

```python
def preco_final(preco, vip):
    total = preco
    if vip:
        desconto = preco * 0.2
        total = total * 0.8 - desconto
    if total < 50:
        total = 50
    return total
```

A suíte atual, com `preco_final(60, True) == 50` e `preco_final(200, False) == 200`, passa com 100% de ramos.

Preço com Desconto VIP

Exercício 3

a) Liste as definições e os usos de `total` e de `desconto`, com o número da linha e o tipo de uso (*c-use* ou *p-use*), e todos os pares du

b) Quais pares a suíte atual não percorre?

c) Escreva, a partir da especificação, o que falta para completar All-Uses. O que acontece, e por que a suíte atual não viu o defeito?

d) Corrija a função e rode a suíte inteira

Síntese

- $V(G)$ dá o tamanho da base, mas caminhos inviáveis e laços com mínimo de voltas mudam o que dá para testar
- Conjuntos MC/DC diferentes acham defeitos diferentes; só o MCC confere todas as combinações
- 100% de ramos não leva cada valor a cada uso: pares du descobertos escondem defeitos mascarados

Simulação e Teste de Software

Laboratório 8: Critérios de Cobertura Estrutural

Prof. Luciano Rossi

Departamento de Ciência da Computação
Centro Universitário FEI

2º Semestre de 2026